
PA0

Syed Shuja Abbas Naqvi

28100250

CS-6304: Advanced Topics in Machine Learning

Abstract

This report takes four widely used vision models, changes one component in each and reports what the change does. On ResNet-152, fine-tuning only the final residual block did better than both head-only and full-backbone training on CIFAR-10, reaching the lowest validation loss (0.254) and the highest validation accuracy (91.7%). Removing the skip connection from two frozen blocks pushed the loss from around 0.5 up to around 1.7, which isolates what the residual path does for the forward pass from its usual role in gradient flow. A Vision Transformer labelled a football (soccer ball) image as “rugby ball” at 87.0% confidence (an ImageNet confusion between two similar ball classes) and its attention heads split up the image in clearly different ways. Masking revealed a sharp split between strategies where the model held 0.761 confidence with half the patches removed at random, but center masking at the same ratio dropped rugby ball to 0.127 and flipped the prediction to soccer ball, the correct label, once the block covered the ball’s center and removed the rugby-like texture. Structured masking over the object therefore changes the prediction much more than when a comparable fraction was removed at random. CLIP hit 96.4% zero-shot on STL-10 with bare class names and aligning the image-text modality gap with orthogonal Procrustes barely moved the number (0.960 against 0.964), since zero-shot classification turns on relative geometry rather than absolute position. A VAE on MNIST with a $d=2$ latent space gave a smooth latent map, while a $d=10$ version reached lower loss and sharper reconstructions. Code: <https://github.com/shujaabbasn/ATML>.

1 Introduction

A vision model can easily be judged by its final accuracy but most of what makes these models work comes down to a few specific design choices and those choices are hard to see when everything is trained end to end and only the top-line number is reported. This report tries to make them visible.

Each task targets a different family. Task 1 looks at ResNet-152 to see what the residual connection does, how features specialize with depth and which transfer-learning strategy works best on CIFAR-10. Task 2 moves to a Vision Transformer and examines its attention maps, how well it holds up when input patches are removed and whether the choice of pooled representation makes a difference. Task 3 covers CLIP, testing how much prompt phrasing changes zero-shot accuracy on STL-10 and measuring the gap between its image and text embedding spaces. Task 4 builds a VAE from scratch on MNIST and compares it against a reference implementation.

Taken together, the four studies give a component-level view of why these models behave as they do, rather than a simple report of headline accuracies alone. Tasks 1 to 3 were run on Google Colab with an NVIDIA T4 GPU after initial runs on a MacBook M1 Pro proved too slow. The small MNIST VAE in Task 4 was trained on CPU (because of GPU unavailability) which is acceptable for a network of this size. All code is available in the repository linked above.

2 Task 1: Inner workings of ResNet-152

2.1 Baseline setup

A pretrained ResNet-152 [1] was loaded with ImageNet weights from torchvision. The final fully connected layer (2048 inputs, originally 1000 outputs for ImageNet) was swapped for a new linear layer with 10 outputs for CIFAR-10. Every parameter in the backbone was frozen so the new head was trainable only. Preprocessing used the model’s own published transform by resizing to 232 on the short side, applying center-crop to 224 and normalizing with ImageNet per-channel statistics (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]).

Adam Optimizer was used with lr = 0.001, batch size 32, and cross-entropy loss. Training ran for 3 epochs.

Table 1: Baseline loss and validation accuracy (frozen backbone, head-only training).

Epoch	Training Loss	Validation Loss	Validation Accuracy
1	0.689	0.520	0.825
2	0.515	0.491	0.841
3	0.480	0.483	0.837

Training loss dropped from 0.689 to 0.480 over the three epochs, and validation loss fell steadily from 0.520 to 0.483. Validation accuracy tracked this, rising from 82.5% to 84.1% by the second epoch. Both curves move down together with no gap opening between them so over these 3 epochs there is no sign of overfitting. Figure 1 shows both curves.

A randomly initialized linear layer on top of frozen features learned to classify CIFAR-10 this well in 3 epochs. This shows that the pretrained features already encode enough visual structure to make the 10 classes roughly linearly separable. Training the full 60-million-parameter network from scratch on 50,000 32×32 images would take far longer and risk overfitting. Freezing the backbone avoids both problems.

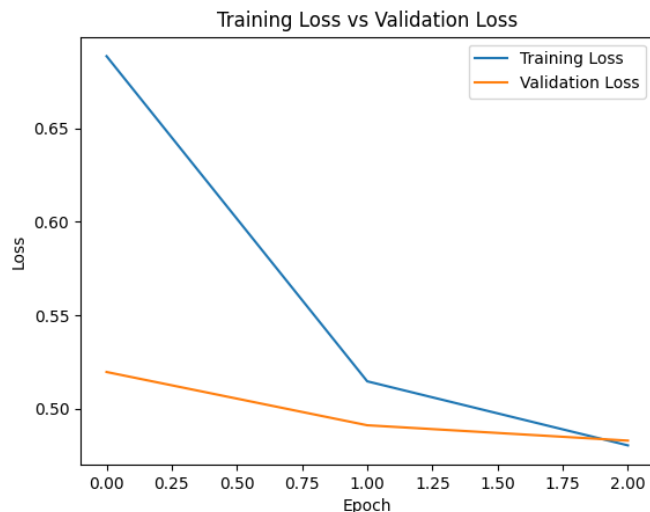


Figure 1: Training and validation loss for the baseline.

2.2 Residual connections in practice

A fresh ResNet-152 was loaded and the forward method of two blocks in layer4 (blocks [1] and [2], both non-downsampling) was replaced with a version that skips the identity addition. The block was rewritten to return its residual branch only which also drops the ReLU that normally follows

the addition. Everything else was kept identical to the baseline (frozen backbone, same head, same optimizer, same epochs).

Table 2: Baseline vs. no-skip model.

Epoch	Base Train	Base Val	Base Val Acc	No-Skip Train	No-Skip Val	No-Skip Val Acc
1	0.689	0.520	0.825	2.024	1.855	0.426
2	0.515	0.491	0.841	1.816	1.747	0.438
3	0.480	0.483	0.837	1.735	1.679	0.453

The gap is large. Without the skip connections, loss sits around 1.7 to 2.0 against the baseline’s 0.5 range, roughly $3.5\times$ worse from removing the skip in just two blocks. Validation accuracy shows the same split, with the no-skip model reaching only about 45% against the baseline’s 84%. Figure 2 shows the two training curves side by side.

It is a forward-pass problem and not a gradient-flow one (no gradient updates these layers) as the backbone was frozen. Each pretrained block was trained to output $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Removing the $+\mathbf{x}$ means the block now outputs only $\mathcal{F}(\mathbf{x})$ which is not what the next layer was trained to receive. The features arriving at the head are corrupted so the head struggles to classify.

In an unfrozen network, skip connections play a secondary role. The gradient through a residual block is $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} (\mathbf{I} + \frac{\partial \mathcal{F}}{\partial \mathbf{x}})$. The $\mathbf{I}$ term gives an unattenuated gradient path. Without it, the gradient must pass through weight layers that can shrink it, causing vanishing gradients in deep networks. This frozen experiment isolated the representational role while a trainable experiment would show both effects stacked on top of each other.

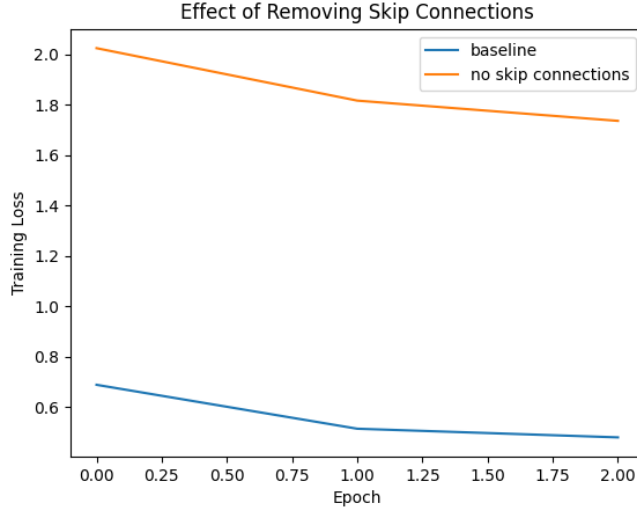


Figure 2: Baseline vs. no-skip training loss.

2.3 Feature hierarchies and representations

Forward hooks were placed on layer1, layer2 and layer4 of a pretrained ResNet-152. 500 test images were passed through and each hook captured that layer’s output. The resulting feature maps were globally average-pooled (averaged over spatial dimensions) to get one vector per image per layer. These vectors were then reduced to 2D with t-SNE and plotted. Here, layer1 stands in for the “early” representation, layer2 for the “middle” one and layer4 for the “late” representation.

An initial run with only 32 images (one batch) produced plots too sparse to read, with about 3 points per class. Increasing to 500 images gave roughly 50 per class which was enough for t-SNE to reveal structure.

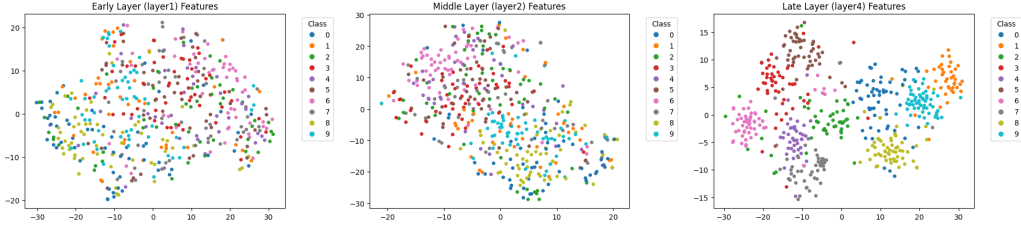


Figure 3: t-SNE of features from layer1 (left), layer2 (center) and layer4 (right).

The layer1 plot is scattered and messy. All ten colours are mixed together with no grouping. By layer2, there are some tendencies with certain colours appearing closer together but heavy overlap remains. In the layer4 plot, distinct clusters appear with same-class points grouping together and different classes pulling apart.

This matches what the feature-hierarchy idea predicts. Early layers learn low-level patterns (edges, textures) that are shared across all classes and do not separate them. Later layers build those into higher-level, class-specific features which is why the clusters only form at depth. The late features encode class identity so a simple linear head on top of them is enough to classify and this explains why transfer learning works.

2.4 Transfer learning and generalization

Four strategies were compared on CIFAR-10:

1. **Head only:** freeze the backbone, train just the linear head. (baseline)
2. **Last block:** freeze everything except layer4 and the head.
3. **Full backbone:** train all parameters, starting from pretrained weights.
4. **Random init:** train all parameters from random weights (no pretraining at all).

All four used Adam ($\text{lr} = 0.001$), 3 epochs, batch size 32.

Table 3: Transfer learning comparison. Best validation loss and accuracy across 3 epochs.

Strategy	Init	Best Val Loss	Best Val Accuracy
Head only	Pretrained	0.483	0.841
Last block	Pretrained	0.254	0.917
Full backbone	Pretrained	0.306	0.896
Random init	Random	0.895	0.693

The last-block strategy got the lowest validation loss (0.254) and the highest validation accuracy (91.7%), beating both the head-only baseline and the full backbone. Full-backbone training, despite updating all 60 million parameters, reached a 0.306 validation loss and 89.6% accuracy which was unexpected as it did worse than tuning the last block alone.

This was likely because the early and middle layers already hold features that work well for CIFAR-10 (edges, textures, shapes) and freezing them preserves those features. The last block is where task-specific adaptation helps most, so unfreezing just that block gives the model flexibility where it matters without disturbing the useful lower-level features. Full-backbone training risks overwriting those lower features and with only 50,000 training images and 3 epochs, there is not enough data or time for the network to recover what it lost.

Random initialization did worst by far (0.895 loss, 69.3% accuracy). Starting from scratch means the model has no useful features to begin with and 3 epochs is nowhere near enough for a 152-layer network to learn good representations from random weights on a small dataset. ImageNet-trained features gave every pretrained strategy a substantial head start which shows the value of pretraining.

The best compute-accuracy tradeoff is the last-block strategy: it cut validation loss nearly in half compared to head-only while only unfreezing a single residual block.

3 Task 2: Understanding vision transformers

3.1 Pre-trained ViT for image classification

A ViT-Base model [2] (patch size 16, input 224×224 , 12 layers, 12 heads, embedding dim 768) was loaded from Hugging Face with attention output enabled so the internal attention weights are returned alongside predictions.

A photograph of a football (soccer ball) on grass was used as the test image. The top-5 predictions were rugby ball (87.0%), soccer ball (12.4%), volleyball (0.2%), baseball (0.03%) and golf ball (0.03%). All five are ball-related classes so the model clearly recognized the object as a ball. The model was surprisingly most confident on “rugby ball” rather than “soccer ball”. This looks like a class-association issue. The prototypical “soccer ball” is the classic white ball with black pentagons and the model may have bound that label mostly to the traditional black-and-white patchwork texture rather than the latest ball. The ball used here is the Adidas Trionda (the 2026 World Cup ball) which is built from only four panels (the fewest of any World Cup ball to date) in a curved, wavy design rather than the usual pentagon-hexagon grid so it carries almost none of that pentagon cue. If the “rugby ball” class leans less on a single fixed texture and more on the general look of a ball on a grass pitch, then a colourful ball sitting on grass could match it more closely than it matches the strict black-and-white soccer-ball template which would explain why the model resolves to rugby ball. The grass background may reinforce this. All of this is plausible reasoning consistent with the confidences observed. For the rest of this section, “the predicted class” refers to rugby ball, the label the model assigned on the unmasked image.

3.2 Attention maps

The attention weights from the final transformer layer were extracted. This layer produces a (1, 12, 197, 197) tensor: 12 heads, each attending across 197 tokens (196 image patches + 1 [CLS] token). The [CLS] token’s attention over the 196 patches was averaged across heads, reshaped from a flat 196-vector into a 14×14 grid (since $224/16 = 14$) and overlaid on the image as a heatmap. The overlay in Figure 4 displays the resized input image and the masking experiment in Section 3.4 operates on the same image after the ViT processor’s resize to 224×224 so both feed the identical pixel tensor to the model.



Figure 4: Averaged [CLS] attention from the final layer.

The averaged map (Figure 4) looks scattered. Attention is spread across the ball and the surrounding grass rather than sitting tightly on the ball alone. The per-head maps below show why.

3.3 Attention head specialization

Each of the 12 heads are plotted separately.

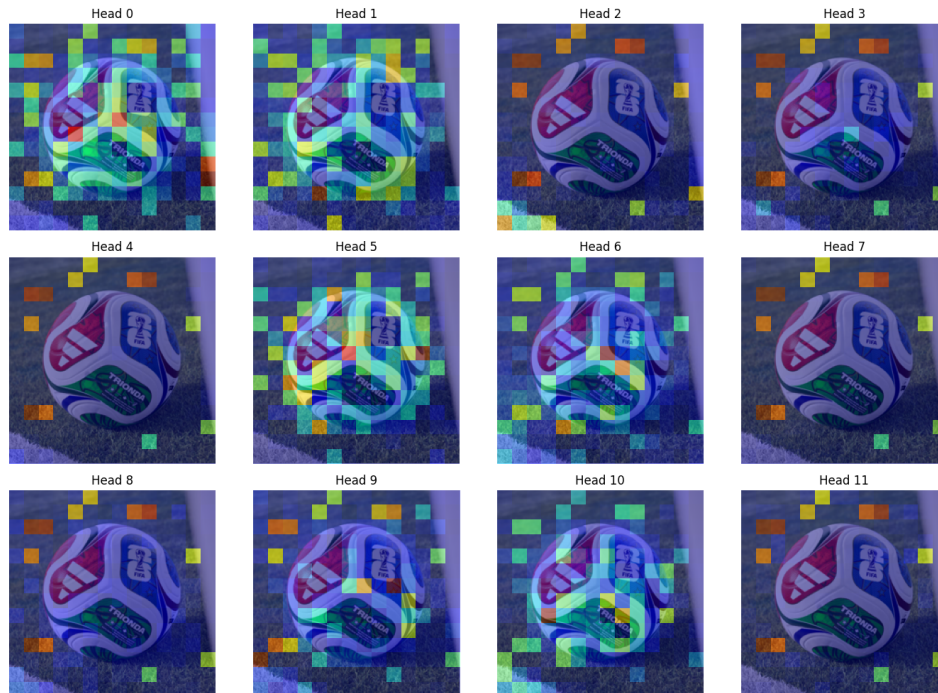


Figure 5: Per-head attention maps from the final transformer layer (Heads 0–11).

The 12 maps look quite different from each other (Figure 5). Heads 0, 1, 5 and 10 spread their attention broadly, covering the ball surface and part of the grass. Heads 2, 3, 4, 7, 8 and 11 focus mostly on scattered background and edge patches (the grass and the white boundary line) rather than the ball itself. This is head specialization where each head attends to different spatial patterns through its own learned projections (Q_h, K_h, V_h) and the averaged map seen in Figure 4 is just the blend of these very different individual maps. Several background-focused heads pull the average off the ball which is why the overlay looked scattered.

This built-in interpretability is one practical difference between ViTs and CNNs. Getting a comparable view out of a CNN usually needs a post-hoc method used after training (CAM only works with a specific global-average-pooling architecture and Grad-CAM needs an extra backward pass reading gradients at a chosen convolutional layer to weight its feature maps). A ViT hands back its attention weights directly from the same forward pass that makes the prediction with no architectural constraint and no extra computation. It also gives one map per head rather than a single combined one. The catch is that attention is not a clean map of what drove the decision as the per-head plots show several heads firing on the background rather than the ball. Attention indicates which patches a head is pulling information from which is not the same as which patches determined the predicted class. A ViT is easier to inspect than a CNN this way but the maps only show where each head is pulling information from, not what actually drove the prediction.

3.4 Patch-masking robustness

Patches were zeroed out at masking ratios of 0.1, 0.3 and 0.5, using both random and center-block strategies. The confidence on rugby ball (the unmasked prediction) and the confidence on soccer ball (the runner-up and correct class) were both tracked so the competition between the two could be followed. As random masking depends on which patches happen to be chosen, each random setting was run over 10 different seeds and the mean is reported. Center masking is deterministic so a single value is given.

The unmasked image was expected to give the highest confidence, with masking only ever lowering it, so light masking would hurt a little and heavy masking a lot. The results only partly match that. The unmasked confidence on rugby ball is 0.870 but at 10% masking it goes up rather than down

Table 4: P(rugby ball) and P(soccer ball) under masking. Random values are the mean over 10 seeds. The std of P(rugby ball) under random masking is 0.045, 0.134 and 0.219 at the three ratios. The unmasked image gives P(rugby ball) = 0.870 and P(soccer ball) = 0.124.

Ratio	Random		Center	
	P(rugby)	P(soccer)	P(rugby)	P(soccer)
0.1	0.942	0.054	0.997	0.002
0.3	0.872	0.121	0.885	0.100
0.5	0.761	0.169	0.127	0.738

(0.942 random, 0.997 center) and only past that does it fall. So masking a small part of the image made the model more confident. Tracking the runner-up class shows why as the soccer ball sits at 0.124 on the unmasked image and at 10% masking it drops (to 0.054 under random masking and 0.002 under center masking) as rugby ball rises. The patches removed at low ratios are mostly background and that background was carrying weak evidence for soccer ball so taking it away sharpens the top class instead of weakening it. The plain “more masking means less confidence” picture only holds once enough of the ball itself starts getting covered which is what the rest of the table shows.

Under random masking, confidence falls gradually as more of the image is removed, from 0.942 at 10% to 0.761 at 50%. The model still identifies the ball from a partial view because random masking leaves patches of the ball scattered across the image for the surviving tokens to aggregate. This is the robustness ViTs are known for and it is the same property that makes masked autoencoder pretraining work where up to 75% of patches are hidden and the model still learns useful representations. The predicted label stayed rugby ball in nine of ten random seeds at 50% masking so scattered removal rarely changes the decision even at high ratios.

Center masking behaves very differently. At low ratios it barely moves the prediction (0.997 at 10%) but at 50% the confidence on rugby ball collapses to 0.127. The center-masked image at 0.5 ratio (Figure 6) shows why as the gray block sits over the middle of the frame and covers most of the ball, leaving only some parts of the ball visible at the edge. Where the confidence goes matters as much as that it drops. At 50% center masking the prediction flips to soccer ball which climbs from 0.124 on the unmasked image to 0.738 and becomes the top class. The center block removes exactly the middle of the ball, where the wave graphics, text and panel pattern sit and those were the cues pushing the model toward rugby ball in the first place. With that texture gone and only the round outline on grass left, the model settles on soccer ball which is the correct label. Heavy center masking removes the texture that was misleading the model and the prediction lands on the correct class as a result.

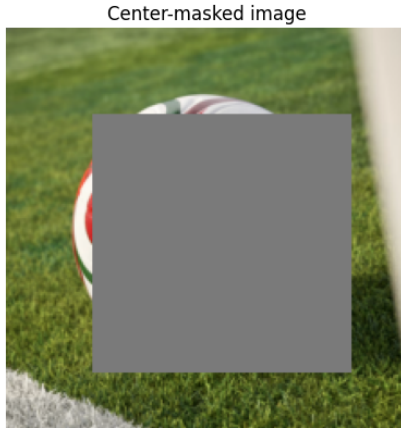


Figure 6: The ball with the central region zeroed (center masking, ratio 0.5). The gray block covers most of the ball, leaving only an edge visible.

The contrast between the two strategies is the key result. Structured (center) masking changes the prediction far more than random masking once the object sits under the masked region, flipping the top class where random masking of a similar fraction leaves it essentially untouched. In this case the flip happened to land on the correct label because the covered region held the texture responsible for the original error but the general point is that a contiguous block over the object moves the prediction decisively while scattered masking of the same size does not.

3.5 CLS token vs. mean-pooled patch tokens

Two feature representations were extracted from the pretrained ViT for 1000 CIFAR-10 train and 300 test images. These were the [CLS] token (the single summary token the model reads for classification) and the mean of all 196 patch tokens. A logistic regression classifier was trained on each.

Table 5: Linear probe accuracy on CIFAR-10.

Pooling	Accuracy
CLS token	0.960
Mean pooling	0.950

Both scored high, with [CLS] slightly ahead. A 1% gap on 300 test images shows that both representations encode class information well.

The small [CLS] edge is likely because this ViT was trained with supervised classification using the [CLS] token as the class summary. The pretraining objective put direct pressure on [CLS] to aggregate class-relevant information. Under a self-supervised pretraining objective that does not single out [CLS], mean pooling might do better because the class information would be distributed across patch tokens rather than concentrated in one. So the “best” pooling depends on how the model was pretrained.

4 Task 3: CLIP

4.1 Zero-shot classification on STL-10

CLIP [3] (ViT-B/32, OpenAI weights) was loaded via the `open_clip` library. The full STL-10 test set (8,000 images, 10 classes) was classified zero-shot under 3 prompting strategies (Plain Labels, Photo and Descriptive).

Table 6: CLIP zero-shot accuracy on STL-10.

Prompt Strategy	Accuracy
Plain labels (“cat”)	0.964
Photo (“a photo of a cat”)	0.959
Descriptive (“a domestic cat with fur and whiskers”)	0.935

All 3 scored above 93%. Plain labels had the highest accuracy which is an interesting result because descriptive or “a photo of a” prompts were expected to work best.

A plausible reason could be that STL-10’s class names are already clean and single-word English nouns (airplane, bird, car, etc.) that CLIP’s text encoder has well-formed embeddings for. Adding “a photo of a” does not sharpen those embeddings. Descriptive prompts scored lowest and this could possibly be because the descriptions were more specific than the class actually is. More words can also narrow the text embedding toward a subset of the class which reduces alignment with the full image distribution.

Plain class names were enough in STL-10 specifically. This pattern may or may not hold on other datasets with more ambiguous class names.

4.2 The modality gap

CLIP image and text embeddings were extracted for 75 STL-10 samples and projected to 2D with t-SNE.

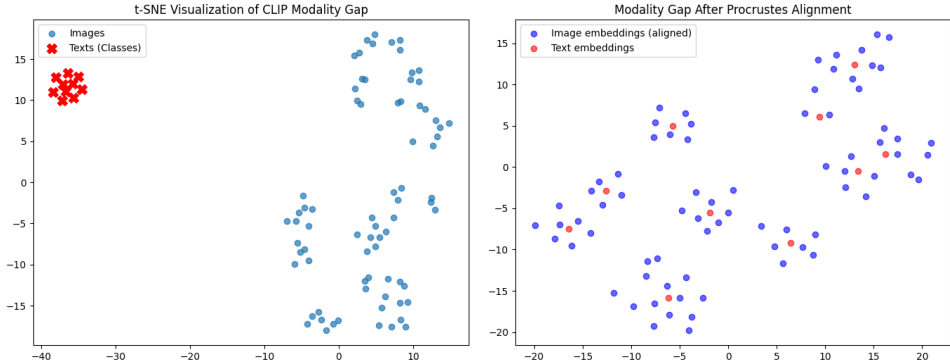


Figure 7: t-SNE of CLIP embeddings before (left) and after (right) Procrustes alignment. Image embeddings in blue, text embeddings in red.

The pre-alignment plot (Figure 7, left) shows image embeddings on one side and text on the other as two clearly separated clusters. This is the modality gap. Even though CLIP was trained to align matching image-text pairs, the two modalities settle into distinct regions of the embedding space.

All embeddings here are L2-normalized. Normalization removes magnitude differences between the two modalities but not directional ones as image and text embeddings still occupy distinct angular regions of the sphere. This is why the gap in Figure 7 (left) persists despite normalization (the separation is directional, not a matter of scale).

CLIP still performs well despite this gap because zero-shot classification depends on relative cosine similarities and not absolute positions. As long as the correct image-text pair has a higher similarity than incorrect pairings within the batch, classification succeeds. A global offset between the two clusters does not change the ranking.

4.3 Bridging the Modality gap

Orthogonal Procrustes alignment was used to rotate the image embeddings toward the text embeddings. The optimal rotation R was computed via the SVD of $X^\top Y$, which factors as $X^\top Y = U \Sigma V^\top$, giving $R = UV^\top$. This finds the rotation that minimizes the Frobenius-norm distance between the two sets of embeddings. Here X holds the normalized image features and Y has the text features of the matching class for each of the 75 samples. The rotation was computed directly from the SVD rather than by calling `scipy.linalg.orthogonal_procrustes`.

After rotation, the t-SNE (Figure 7, right) shows the image and text clusters closer together but the zero-shot classification accuracy shifted from 0.964 (plain labels, pre-alignment) to 0.960 which is nearly identical.

This is because the Procrustes rotation closes the geometric gap but the similarity rankings were already correct before alignment (what CLIP was trained for) and rotating the embeddings does not improve those rankings. The small accuracy dip (0.004) may reflect the rotation being fit on only 75 paired samples so it slightly disturbs the fine similarity structure on the rest of the test set rather than sharpening it.

5 Task 4: VAE

5.1 Building the VAE

A VAE was built in PyTorch for MNIST (28×28 grayscale digits, pixel values in $[0, 1]$). The architecture:

- **Encoder:** linear layer ($784 \rightarrow 500$, ReLU), then two parallel linear layers outputting μ (mean) and $\log \sigma^2$ (log(variance)).
- **Reparameterization:** $\mathbf{z} = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This moves the randomness into ϵ so gradients can flow through μ and σ into the encoder.
- **Decoder:** linear layer (latent dim $\rightarrow 500$, ReLU), then linear to 784 with sigmoid output, treating each pixel as a Bernoulli probability.
- **Loss:** binary cross-entropy summed over pixels (reconstruction) plus the closed-form Gaussian KL: $D_{KL} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$. This is summed over the batch.

Training: 20 epochs, Adam (lr = 1e-3), batch size 128. One model was trained with latent dimension $d=2$ (for direct 2D visualization) and one was trained with $d=10$ (for comparison).

5.2 Results

5.2.1 Latent space

With $d=2$, the encoder means $\mu(\mathbf{x})$ for the full test set were plotted directly in 2D.

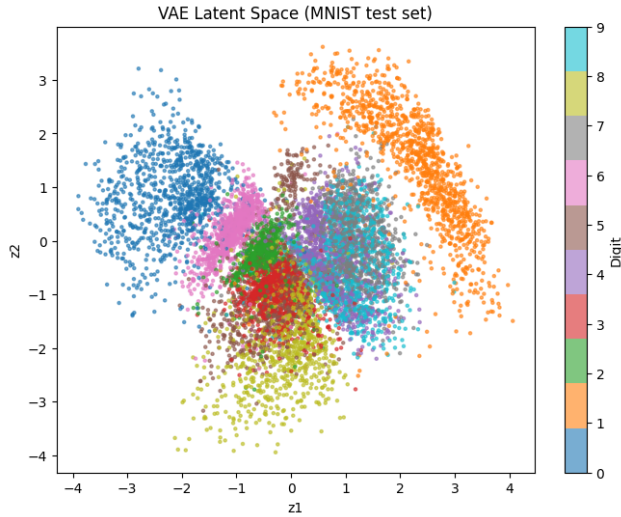


Figure 8: 2D latent space of the $d=2$ VAE on MNIST test images. Colored by digit.

Same-digit points cluster together. There is overlap between visually similar digits (like 4,9 and 3,8). The space is smooth and continuous with no large empty holes. That smoothness comes from the KL term which pushes the encoder’s output distribution toward the standard normal prior and prevents it from placing codes in disconnected regions.

5.2.2 Reconstructions

Test images were encoded, sampled and decoded. Originals and reconstructions are shown side by side below.

The reconstructions generally capture each digit’s shape and identity but lose fine detail. The lines are blurrier than the originals and with the heavy $d=2$ bottleneck some digits drift toward a similar-looking neighbour (the sampled 7 and 4 in Figure 9 decode closer to a 9). The KL regularization forces the decoder to produce outputs that work for a range of nearby latent codes which smooths out sharp details and 2 latent dimensions leave little room to separate confusable digits.

5.2.3 Samples from the prior

Random 2D vectors were drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and decoded.

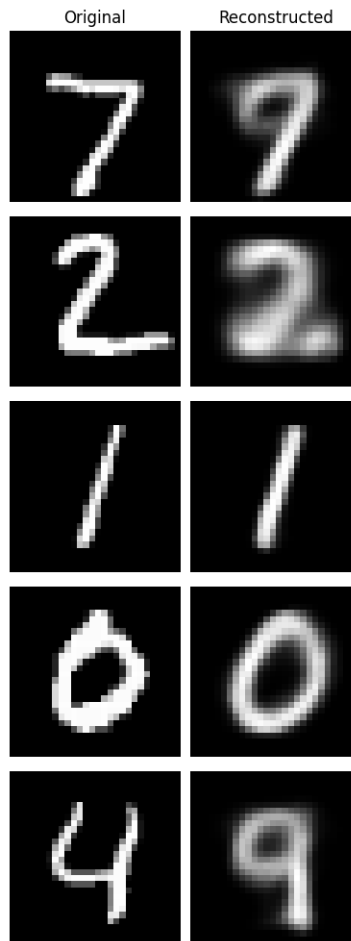


Figure 9: Originals (left) and reconstructions (right) from the $d=2$ VAE.

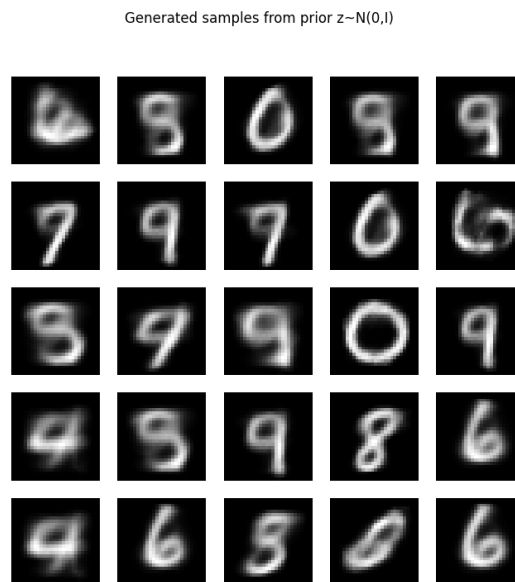


Figure 10: Digits generated by sampling from the prior.

Most samples are recognizable digits. Some are ambiguous, looking like a blend of two classes (e.g. 4 and 9). These correspond to the boundary regions in the latent space where clusters overlap. The fact that random samples from the prior produce plausible outputs at all confirms that the latent space is well-structured since the KL term ensured the encoder’s distribution covers the prior region and sampling from it lands in meaningful territory.

5.3 Doersch comparison

Doersch’s VAE tutorial [4] from the reading list uses a similar setup for MNIST.

Architecture: both are MLP-based. This implementation has 500-neuron hidden layers.

Loss: Doersch models pixels as Bernoulli outcomes and uses sigmoid cross-entropy which is the same as the BCE reconstruction loss used here. The KL term is the same closed-form Gaussian expression in both.

Latent dimension: the models here used $d=2$ and $d=10$. Doersch notes that VAE performance is “insensitive to the dimensionality of z ” across a broad range. These results showed that over 20 epochs, the two sizes had not converged to the same loss so the dimension still mattered here.

Table 7: Loss by latent dimension (epoch 1 vs. epoch 20).

d	Starting Loss	Final Loss
2	186.8	150.2
10	156.9	106.7

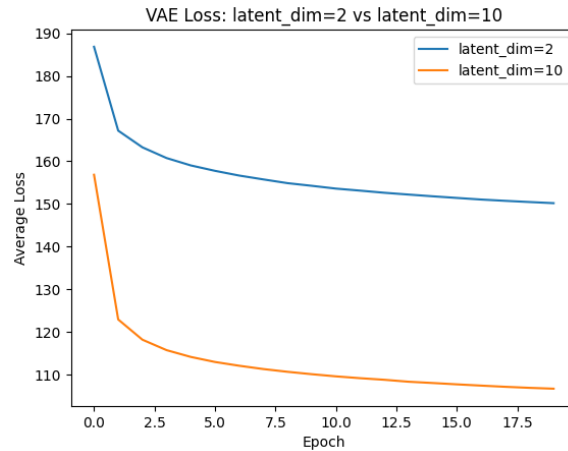


Figure 11: Training loss over 20 epochs for $d=2$ and $d=10$.

The $d=10$ model reached notably lower loss. Its reconstructions were also sharper since 10 latent dimensions can encode more detail than 2. The $d=2$ model’s higher loss is the cost of compressing 784 pixels into 2 numbers which is a heavy bottleneck. However, $d=2$ is what makes the latent-space plot in Figure 8 possible so both serve a purpose. Doersch’s “insensitivity” claim is about the final quality of a well-converged model and the gap here reflects that 20 epochs was not enough for the smaller bottleneck to close its larger reconstruction error.

Generated samples: Doersch reports that most of his generated digits look realistic, with a number of ambiguous in-between cases. The outputs here match that description. Ambiguous cases tend to involve visually similar pairs (like 4,9 and 3,8) where the latent-space clusters overlap.

6 Conclusion

A few of these results only make sense once the component is looked at on its own. The findings below follow the order of the tasks.

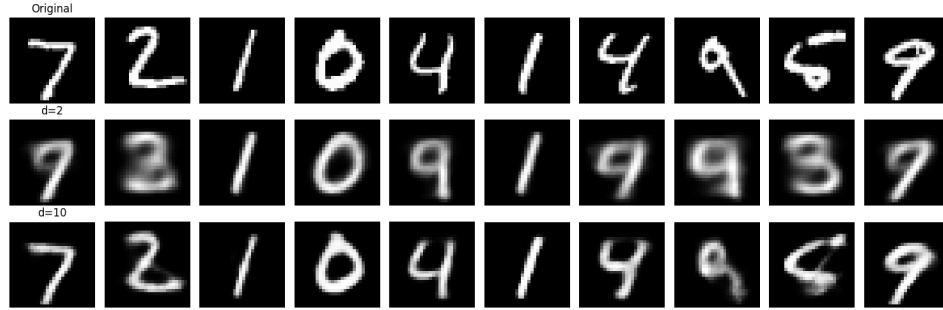


Figure 12: Reconstructions from the $d=2$ (middle row) and $d=10$ (bottom row) VAEs, with originals on top.

Task 1 (ResNet-152). The residual connection mattered even with a frozen backbone where gradient flow is not a factor. Removing it still corrupted the features because each block was trained to output $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ and dropping the $+\mathbf{x}$ hands the next layer an input it was never trained to receive. Here only the representation effect is observed, not the gradient effect. The transfer-learning comparison was the other surprise, since fine-tuning only the last block beat full-backbone training on both validation loss and accuracy. The lower layers’ pretrained features already suited CIFAR-10 and were better left alone.

Task 2 (ViT). The attention heads specialized visibly, each attending to different regions rather than one shared map. Masking showed a clear split between the two strategies. The model stayed robust to random masking, holding 0.761 confidence with half the patches removed but center masking at the same ratio dropped rugby ball to 0.127 and flipped the prediction to soccer ball (0.738), the correct label, once the block covered the ball’s center and removed the rugby-like texture. A contiguous block over the object therefore changes the prediction far more than when a comparable fraction was removed at random. The model also classified the unmasked ball as “rugby ball” rather than “soccer ball,” a known confusion between two visually similar ImageNet classes.

Task 3 (CLIP). Zero-shot classification worked best with plain class names rather than engineered prompts and the image-text modality gap did not hurt accuracy. Closing that gap with a Procrustes rotation left accuracy nearly unchanged which fits the fact that classification depends on relative similarity rankings, not the absolute positions the gap describes.

Task 4 (VAE). The VAE traded sharpness for a continuous latent space and raising the latent dimension from 2 to 10 bought back some of that detail. The two sizes hadn’t converged to the same loss after 20 epochs. Doersch’s claim of insensitivity to latent dimension is about converged models and 20 epochs likely wasn’t enough to get there.

Two patterns can be seen across all the tasks. The first is that the headline effect of a component had a different outcome once it was tested in isolation. The last block, not the full backbone, gave the best transfer, center masking corrected the prediction rather than simply degrading it and closing the modality gap did nothing for accuracy. The second is that several of these conclusions only hold at the small scale used here. The masking and attention work is based on one image, the Procrustes fit used 75 paired samples and both the ResNet fine-tuning and the VAE trained for only a handful of epochs. So the trends here are better treated as clean demonstrations of each mechanism on a small scale than as final numbers.

References

- [1] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [2] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [3] OpenAI. CLIP: Connecting text and images. <https://openai.com/index/clip/>, 2021.
- [4] C. Doersch. Tutorial on variational autoencoders. *arXiv:1606.05908*, 2016.